TRACK 2 · FINANCE · 1-DAY HACKATHON

Personal Investment Research Agent



A RAG + tool-calling ReAct agent that answers from financial documents and live market data, and always tells you which source a number came from.

Team Krazy Coder

Tshewang Kinley Rigzin · Adesh Singh · Jigme Tenzin

Why this agent is needed

Documents are accurate but old

Annual reports and fact sheets hold audited figures, but only for a past period.

Live data is fresh but bare

A price or headline has no context: no revenue, no profit, no definition.

Units cause wrong answers

INR vs USD and crores vs millions: one mix-up makes the whole answer wrong.

Goal: one agent that knows which source to trust for which question.

What the agent does

LIVE market data

- Latest share price (with currency)
- Recent news headlines per ticker
- Source: Yahoo Finance via yfinance

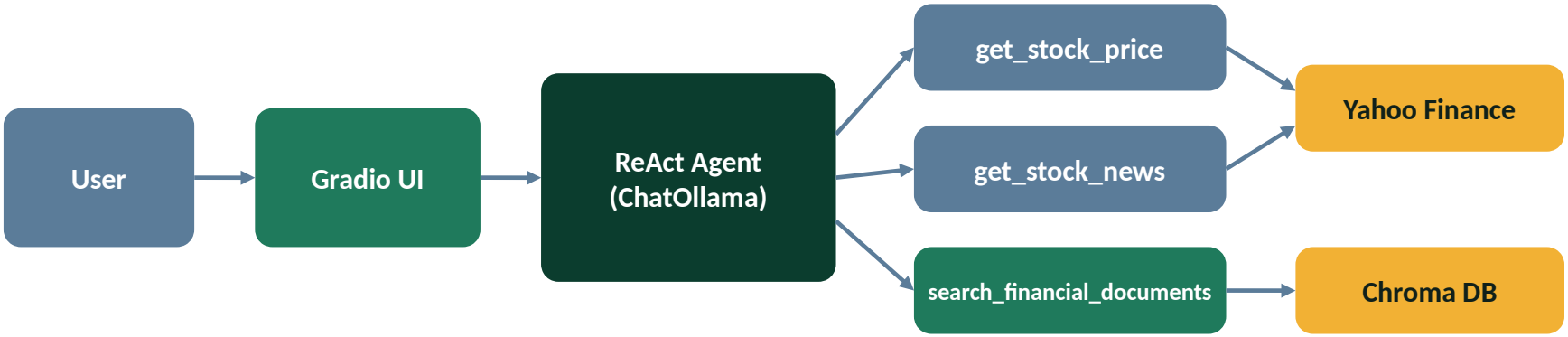
STATIC document data

- Revenue, profit, ratios as reported
- Fund fact sheets and investing guides
- Source: PDFs retrieved from Chroma (RAG)

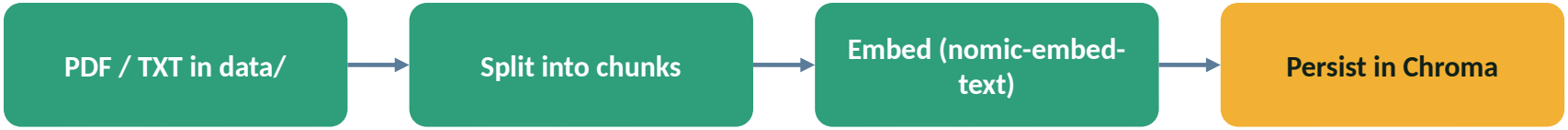
ONE grounded answer: every number labelled (live market) or (annual report, page n) with its unit.

System architecture

ONLINE: answering a question



OFFLINE: building the index (ingest.py)



Tech stack

Ollama + ChatOllama

Local LLM, no API cost. Needs a model with tool-calling support.

LangChain 1.0

Loaders, splitters, @tool, and create_agent for the ReAct loop.

Chroma

Persistent vector store used by the retriever.

nomic-embed-text

Embedding model that turns text chunks into vectors.

yfinance

Free Yahoo Finance access for live price and news.

Gradio

Chat UI with the required disclaimer, easy to share.

Steps 1-2: LLM and loading

Get ChatOllama working first

Test one plain call before adding anything else. Temperature 0 keeps numeric answers stable.

PyPDFLoader per file

Returns one Document per page, with source and page number saved as metadata.

Why metadata matters

The page number lets every answer cite exactly where a figure came from.

```
# agent.py
llm = ChatOllama(
  model="llama3.1",
  temperature=0)

# ingest.py
docs += PyPDFLoader(
  path).load()
# -> Document(page_content,
#    metadata={source, page})
```

Step 3: split without breaking numbers

Chunk too small

```
Chunk 1:  Revenue from operations
Chunk 2:  9,15,000   (INR crore)
```

The label and its number land in different chunks. The retriever returns a number with no meaning.

Chunk 1200, overlap 250

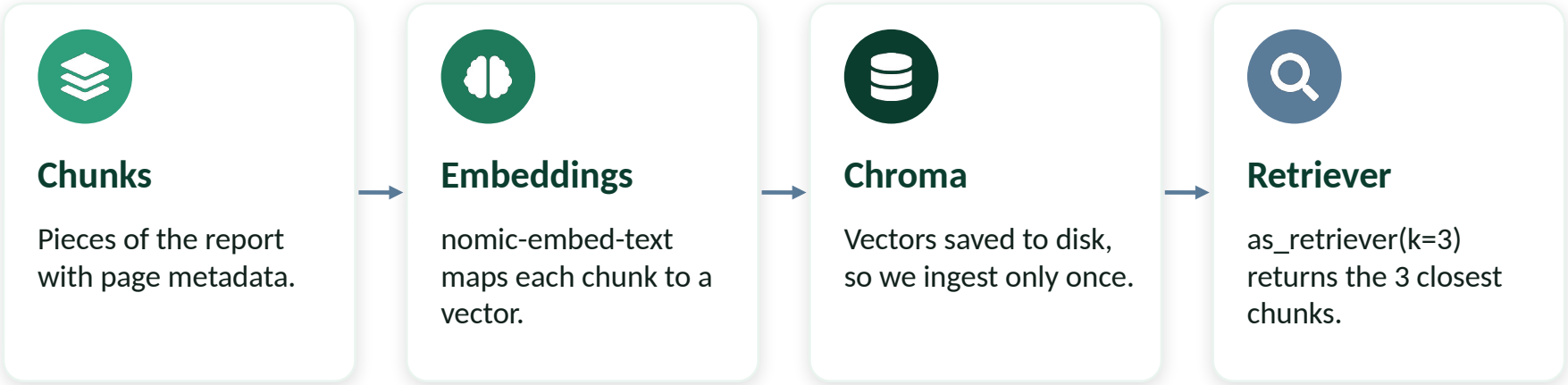
```
Chunk 1:  Revenue from operations
          9,15,000   (INR crore)
```

Rows stay with their headers and units. Overlap protects values near a boundary.

```
RecursiveCharacterTextSplitter(
  chunk_size=1200, chunk_overlap=250,
  separators=["\n\n","\n",". "," ",""])
```

Tune and verify: print chunks around tables, then change the two numbers until labels and values stay together.

Steps 4-5: embed, store, retrieve



Test before moving on

python ingest.py --test "what was revenue in Q2" must return the correct page.

Steps 6-7: custom tools

LIVE

`get_stock_price`

In: ticker such as RELIANCE.NS or AAPL.

Out: price per share with currency (INR for .NS/.BO, USD for US).

LIVE

`get_stock_news`

In: same ticker format.

Out: up to 5 recent headlines with publisher.

STATIC

`search_financial_documents`

In: a specific query, e.g. net profit FY24.

Out: top 3 passages with file, page; document's own units.

Docstrings state units clearly, and every result starts with a [SOURCE: LIVE ...] or [SOURCE: STATIC ...] tag.

Step 8: the ReAct agent

THE LOOP

Reason
Live, static, or both?

↓

Act
Call a tool with arguments

↓

Observe
Read the output, loop or answer

SYSTEM PROMPT RULES

1. **Classify**
   Does the question need live data, document data, or both?
2. **Call tools**
   Use the right tool(s). Both when needed.
3. **Combine**
   Merge results into one answer. Never invent numbers.
4. **Label**
   State source and unit for every number. Add the disclaimer.

Example: a question needing both

USER	Compare reported net profit with today's price of RELIANCE.NS
THOUGHT	Needs document data AND live data, so call both tools
ACTION 1	search_financial_documents(net profit) -> STATIC, INR crore
ACTION 2	get_stock_price(RELIANCE.NS) -> LIVE, INR per share
ANSWER	Each number tagged with source, unit and period
GUARD	Ends with: not personalized financial advice

Step 9: Gradio deployment



gr.ChatInterface

Chat history is passed to the agent on every turn.

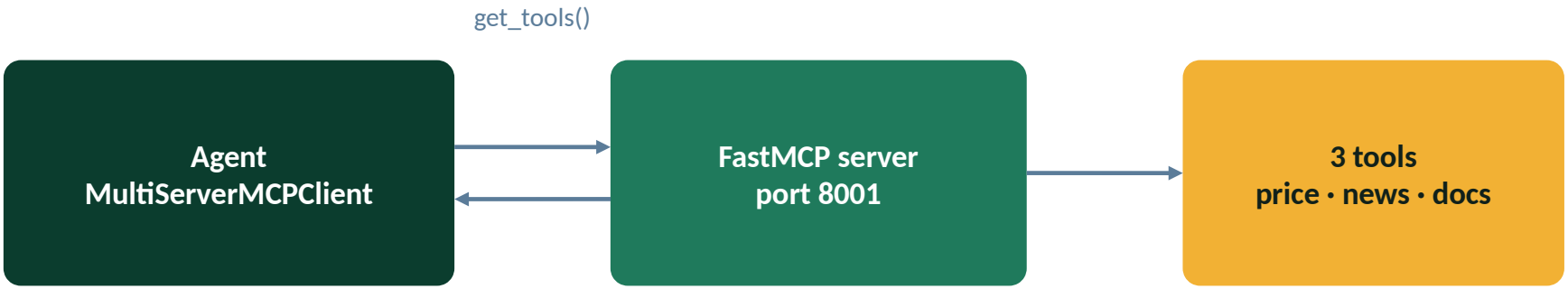
Visible disclaimer

Shown at the top and bottom, as the rules require.

One command

python app.py opens the app at localhost.

Bonus: tools over MCP



Run the server

python mcp_server.py exposes the same @tool functions over streamable HTTP.

Fetch at runtime

python agent_mcp.py discovers tools by URL, not by import.

Why it matters

Tools become reusable by any MCP-compatible client or agent.

How we meet the judging criteria

Live vs static distinction

Source tags in tool output + prompt rule: label every number as live market or document.

Numeric accuracy from retrieval

Tuned chunking keeps labels with values; retriever tested on numeric queries; temperature 0.

Correct tool sequencing

Prompt forces classify, then call one or both tools, then combine into one answer.

Challenges and next steps

Challenges solved

- Tables split mid-row: larger chunks, overlap, better separators
- Unit confusion: units in every docstring and answer
- Tool-call reliability: pick a tool-capable model, temperature 0

Next steps

- Table-aware PDF parsing for cleaner numbers
- A small test set of numeric questions to score accuracy
- More tools: ratios, currency conversion, portfolio view

Python program: files and run

```
invest_agent/
  config.py       settings
  ingest.py       load, split, embed
  tools.py        3 @tool functions
  agent.py        ReAct agent
  app.py          Gradio UI
  mcp_server.py   bonus: server
  agent_mcp.py    bonus: client
  data/           your PDFs
  requirements.txt
```

```
pip install -r requirements.txt
ollama pull llama3.1
ollama pull nomic-embed-text
# put a PDF in ./data, then:
python ingest.py
python ingest.py --test "revenue Q2"
python agent.py
python app.py
# bonus (two terminals)
python mcp_server.py
python agent_mcp.py
```

Code: ingest.py (load and split)

```
def load_documents():
    docs = []
    for path in glob.glob(os.path.join(config.DATA_DIR, "*")):
        if path.lower().endswith(".pdf"):
            docs += PyPDFLoader(path).load()     # 1 Document per page + metadata
        elif path.lower().endswith((".txt", ".md")):
            docs += TextLoader(path, encoding="utf-8").load()
    return docs

def build_index():
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1200, chunk_overlap=250,
        separators=["\n\n", "\n", ". ", " ", ""])  # prefer row/paragraph breaks
    chunks = splitter.split_documents(load_documents())
    Chroma.from_documents(chunks, OllamaEmbeddings(model="nomic-embed-text"),
        collection_name="financial_docs", persist_directory="chroma_db")
```

Steps 2-4. Run once with: python ingest.py

Code: ingest.py (retriever)

```
def get_vectorstore():
    return Chroma(collection_name=config.COLLECTION,
        embedding_function=OllamaEmbeddings(model=config.EMBED_MODEL),
        persist_directory=config.CHROMA_DIR)

def get_retriever():
    return get_vectorstore().as_retriever(search_kwargs={"k": 3})

def test(query):          # python ingest.py --test "what was revenue in Q2"
    for i, d in enumerate(get_retriever().invoke(query), 1):
        print(f"[{i}] {d.metadata.get('source')} p.{d.metadata.get('page')}")
        print(d.page_content[:500])
```

Step 5. Simplified excerpt. The test function is the check before building the agent.

Code: tools.py (live data)

```
@tool
def get_stock_price(ticker: str) -> str:
    """LIVE MARKET DATA. Latest traded price for a ticker.
    NSE '.NS' / BSE '.BO' -> INR; US tickers (AAPL) -> USD.
    Per share, NOT in crores/millions."""
    fi = yf.Ticker(ticker).fast_info
    return (f"[SOURCE: LIVE MARKET] {ticker.upper()} last price: "
            f"{fi['last_price']:,.2f} {fi['currency']} per share.")

@tool
def get_stock_news(ticker: str) -> str:
    """LIVE MARKET DATA. Up to 5 recent headlines for a ticker
    (same ticker format as get_stock_price)."""
    items = yf.Ticker(ticker).news[:5]
    lines = [f"- {n.get('content', n).get('title')}" for n in items]
    return "[SOURCE: LIVE NEWS]\n" + "\n".join(lines)
```

Steps 6-7. Simplified excerpt; the project file also handles errors. Docstrings state currency and units.

Code: tools.py (documents)

```
@tool
def search_financial_documents(query: str) -> str:
    """STATIC DOCUMENT DATA. Top 3 passages from the ingested annual reports
    and fact sheets, with file name and page. Figures are in the document's
    own units (e.g. INR crores) and period, NOT today's values."""
    docs = get_retriever().invoke(query)
    out = ["[SOURCE: STATIC DOCUMENT]"]
    for d in docs:
        out.append(f"(file: {os.path.basename(d.metadata['source'])}, "
                   f"page: {d.metadata.get('page', '?')})\n{d.page_content}")
    return "\n\n---\n\n".join(out)

TOOLS = [get_stock_price, get_stock_news, search_financial_documents]
```

Step 6b. Wraps the retriever as a tool. TOOLS is imported by the agent.

Code: agent.py (ReAct agent)

```
SYSTEM_PROMPT = """You are a personal investment research assistant.
For EVERY question:
1. Decide: LIVE data, STATIC document data, or BOTH.
2. Call get_stock_price / get_stock_news (live) and/or
   search_financial_documents (documents). Both when needed.
3. Combine into ONE grounded answer. Never invent numbers.
4. Label every number with source + unit:
   "(live market, INR per share)" or "(annual report, p.X, INR crores)".
5. No personalized advice. End with: "This is not personalized financial advice."
"""

llm = ChatOllama(model="llama3.1", temperature=0)
agent = create_agent(model=llm, tools=TOOLS, system_prompt=SYSTEM_PROMPT)

def ask(messages):
    return agent.invoke({"messages": messages})["messages"][-1].content
```

Step 8. Prompt shortened for the slide; the full version is in agent.py.

Code: app.py (Gradio)

```
import gradio as gr
from agent import ask
DISCLAIMER = "Not personalized financial advice. Educational use only."

def respond(message, history):
    msgs = [{"role": h["role"], "content": h["content"]} for h in history]
    msgs.append({"role": "user", "content": message})
    return ask(msgs)

with gr.Blocks(title="Investment Research Agent") as demo:
    gr.Markdown("# Personal Investment Research Agent")
    gr.Markdown(DISCLAIMER)
    gr.ChatInterface(respond, type="messages")
    gr.Markdown(DISCLAIMER)

demo.launch()
```

Step 9. Launch with: python app.py

Code: MCP server and client

```
# mcp_server.py
mcp = FastMCP("finance-tools", port=8001)
for t in (get_stock_price, get_stock_news, search_financial_documents):
    mcp.tool(name=t.name, description=t.description)(t.func)
mcp.run(transport="streamable-http")

# agent_mcp.py
client = MultiServerMCPClient({"finance": {
    "url": "http://localhost:8001/mcp", "transport": "streamable_http"}})
tools = await client.get_tools()
agent = create_agent(ChatOllama(model="llama3.1", temperature=0), tools,
                     system_prompt=SYSTEM_PROMPT)
res = await agent.ainvoke({"messages": [{"role": "user", "content": question}]})
```

Bonus. Server on port 8001; the agent loads the tools at runtime. Simplified excerpt.

Thank you!



Live demo: compare reported net profit with today's price of RELIANCE.NS

Team Krazy Coder

Tshewang Kinley Rigzin · Adesh Singh · Jigme Tenzin

Educational use only. Not personalized financial advice.